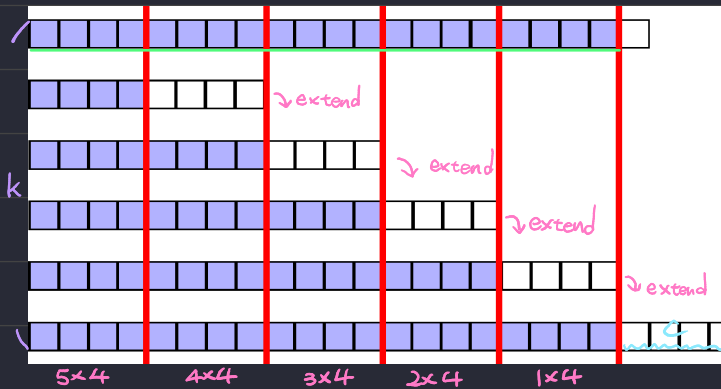


CH3 Arrays, Stacks, and Queues

Arrays

Incremental Strategy Analysis (Adding c everytime)



$$k = \frac{n}{c}$$

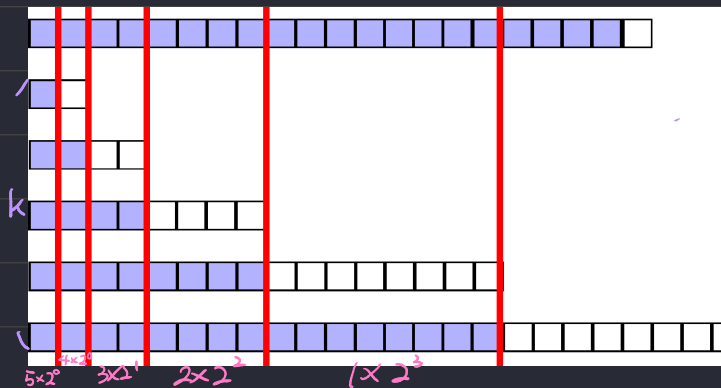
$$\text{total operations} = \underbrace{n}_{\text{push}} + \underbrace{\text{sum(blocks)}}_{\text{copy}}$$

$$= n + c \cdot \frac{(1+k) \cdot k}{2}$$

$$= n + \frac{c}{2} \cdot (k + k^2)$$

$$= O(n)$$

Doubling Strategy Analysis



Array-based Stacks (FILO)



Performance

- The space used is $O(n)$
- Each operation (push, pop) runs in $O(1)$

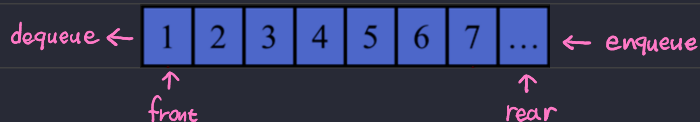
Limitations

- Maximum size of the stack must be defined
→ sol: notify user when it's full / use extensible array

Useful for solving...

- Reversing array (LIFO) $[1, 2, 3, 4, 5] \rightarrow [5, 4, 3, 2, 1]$
- Matching Parentheses $()()(())\{([()])\}$

Array-based Queue (FIFO)



If we left shift every element after a dequeue, it might be not efficient $O(n)$

→ Use **circular array** to solve it. (When front/rear hits the end, goto $A[0]$)

DO NOT use $f=r$ to consider an array is empty or not, use $size == (N - f + r) \% N$

Performance

- The space used is $O(n)$
- Each operation takes $O(n)$

Limitations

- Maximum size of the queue must be defined
→ sol: notify user when it's full / use extensible array

Useful for solving...

Round Robin Schedulers (輪轉問題)

Double Ended Queues

An queue that support insertion and deletion at both front and rear

e.g. insertFirst(e), insertLast(e), removeFirst(), removeLast()

Case Study: Polish Notation

Like binary tree, you can write operator as root, operand as left and right

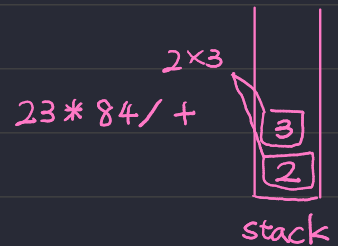
Infix	Postfix	Prefix	Notes
$A * B + C / D$	$A B * C D / +$	$+ * A B / C D$	multiply A and B, divide C by D, add the results
$A * (B + C) / D$	$A B C + * D /$	$/ * A + B C D$	add B and C, multiply by A, divide by D

Ex: Infix Postfix
 $7-2 \times 3$ $7 2 3 \times -$
 $(7-2) \times 3$ $7 2 - 3 \times$
parenthesis-free ↑

Evaluating Postfix Expressions

Use stack, move number we meet into stack, calc last two element if operator seen

```
01 Initialize a stack S
02 Scan the expression from left to right character by character
03 if (character c is an operand)
04   push(S, c)
05 else // c is an operator
06   a = pop(S)
07   b = pop(S)
08   calculate the result as value
09   push(S, value)
10 return pop(S)
```



Infix to Postfix